Lichen over BABE

withdrawals of a variable amount, by authenticated request

a design note with cost estimates

waxwing

Version 0.1, October 2026

a companion to *Lichen*, version 0.2 [1]

Abstract

BABE [3] settles a Groth16 proof on Bitcoin in three transactions, using witness encryption and a small garbled circuit, but its statement, public inputs included, is fixed when the parties set up. A lichen withdrawal is for an amount b that Alice chooses only when she withdraws. We describe how BABE can serve as lichen's on-chain gadget without any change to its cryptography: the amount leaves the public input and enters the witness, together with the hub's one-time signature on it, which the hub must give on chain for exactly the amount Alice asserted there, or lose. We sketch why this binds the proved amount to the paid one, give the transaction graph, and estimate costs from BABE's published measurements and lichen's: a refused withdrawal of about 31 kvB in six transactions against BitVMX's 100–212 kvB in 23–25, a hub able to hold, for a dispute of a given length, about seven times as many contracts before congesting the chain pays (three without soldering), a tower for Alice that holds none of her keys, since her only move due in a window is a hash preimage, and a setup per contract of about 14 minutes of computation with BABE's soldering, or, we estimate, about 40 seconds with verifiable secret sharing in its place, and 90–220 MiB of storage for the user. Nothing here is built, and the security argument is a sketch.

Contents

1 Status and scope **2**

2 BABE in brief **2**

3 Where BABE fits in lichen **3**

4 The fixed statement **4**

5 Authenticated amounts **4**

5.1 The statement . . . 4
5.2 The flow . . . 5
5.3 Why the amount is bound . . . 5
5.4 The guest . . . 6
5.5 Security, as a sketch . . . 6

6 The transaction graph **6**

7 Alice's liveness: a keyless tower **8**

8 Estimates **9**
8.1 Transactions and disputes 9
8.2 Congestion 10
8.3 Reserves and time 11
8.4 Setup per contract 11
8.5 Setup by verifiable secret sharing 11

9 With a counter-proof **13**

10 Open questions **14**

1. Status and scope

The main paper [1] treats the check of a disputed withdrawal proof as a gadget with a narrow interface: given a proof whose public input binds the program and its journal, adjudicate a two-party claim on Bitcoin, with the contract's fixed terms checked, the amount b readable by Script for a variable payout, and the loser paying. The gadget built today is BitVMX's interactive search [10], described and measured in a companion [2]. This note describes a second gadget, BABE, which is not built.

What is ours here is narrow: the binding of a late-chosen amount by the hub's authentication (Section 5), first put forward in a discussion on Delving Bitcoin [9], and the cost estimates of Section 8. BABE, its witness encryption and its garbled circuit are its authors' [3, 4, 5]; the problem of public inputs that are known only at proving time has been treated at the cryptographic layer by GOAT's researchers [8]. Figures marked (BABE) are the BABE paper's measurements, taken from its revised version (October 2026), whose on-chain figures are larger than those of the January version; figures marked (lichen) are measured in lichen's proof of concept; the rest are our estimates.

2. BABE in brief

Groth16's verification, written additively in the target group, is

$$e(\pi_1, \pi_2) + e(\pi_3, \delta) = T, \qquad T = e(\alpha, \beta) + e(L, \gamma), \qquad L = IC_0 + \textstyle\sum_i s_i \, IC_i,$$

where π_1, π_2, π_3 are the proof, s_i the public inputs, and the rest the verifying key. Witness encryption (WE) lets a party encrypt a message so that only a holder of a valid proof can decrypt it. For pairing relations *linear* in the witness there is a simple scheme [4]: for a secret scalar r, publish r times the public elements; a holder of the witness pairs it against them and obtains $r\,T$, which unmasks the message. Groth16's check is not linear in the witness, because of $e(\pi_1, \pi_2)$. BABE's idea is to supply $r\pi_1$ by two-party computation once π_1 is known:

- *Setup.* The verifier picks r and a message *msg*, publishes the WE ciphertext ($r\delta$ and *msg* masked by $r\,T$; 480 bytes), commits to *msg* by its hash, and garbles a circuit computing $r\pi_1$ from the bits of π_1, with r inside. The circuit is small because group addition to a fixed

point is linear in a six-term feature vector of the point, so all but a point-validation gadget is a decomposable randomized encoding with no garbled tables, after Argo MAC [5]: 22 MiB per instance, 175 ms to make (BABE).

- *Assert.* The prover posts π_1 under a Lamport signature (508 bits).
- *ChallengeAssert.* To contest, the verifier must post the circuit's input labels for exactly those bits; a script checks that its labels and the prover's Lamport signature sign the same value. (A garbled circuit's input labels *are* a Lamport signature of its input.)
- The prover evaluates the circuit, obtains $r\pi_1$, completes the decryption with π_2 and π_3, and, if the proof is valid, publishes *msg* (*WronglyChallenged*), which blocks the verifier's *Failure*; after a timelock Δ_2 the prover's *Success* pays. If the verifier never challenges, Success pays after Δ_2 anyway. Failure needs Δ_1 after the challenge, and $\Delta_2 > \Delta_1 + 2u$ for a liveness bound u.

On chain, a contested valid proof costs Assert 9,240 vB, ChallengeAssert 17,400 vB and WronglyChallenged 149 vB (BABE), with one set of labels. Against a verifier who might garble wrongly (one whose circuit never lets the prover decrypt), the prover checks the garbling by cut-and-choose: for example 2,268 instances of which 4 are kept, 1 min 37 s and 1.8 GB at peak on a 16-thread laptop (BABE). Without more, the verifier must then post 4 sets of labels; *soldering*, a SNARK proving the kept instances' labels consistent, brings it back to one at about 12 minutes more of setup (BABE); verifiable secret sharing could do the same in seconds (Section 8.5). The WE is extractable, proved in the generic group model. The protocol is proved robust (an honest prover is paid) and knowledge-sound (a prover who is paid knows a witness) for a setup that both parties run honestly. Against a verifier who garbles wrongly, cut-and-choose is argued, failing with probability at most 2^{-40} for the parameters above, but not proved (BABE).

3. Where BABE fits in lichen

The proving stack is unchanged. lichen's withdrawal proof is already a Groth16 proof over BN254: RISC Zero's recursion compresses the guest's STARK receipts into a succinct receipt, and RISC Zero's shrink-wrap proves that receipt's verification in Groth16, with RISC Zero's single trusted setup [11]. BABE verifies exactly such a proof. The guests, the STARKs, the sequencer's chain proofs and the wrap all stay as they are; only the statement's journal changes (Section 5).

The roles. Alice, who claims her withdrawal, is BABE's prover; the hub, the only party a false claim can rob, is its verifier, and so the garbler. BABE is a protocol between one prover and one verifier, both known at setup, which is what a lichen contract is: no committee, no open set of challengers, and no one else's funds in the output. Designs whose challengers are not known in advance pay heavily to approximate this; a bilateral contract has it for free.

Setup is per contract, and two-party. The hub garbles for each contract; Alice checks the garbling by cut-and-choose and keeps the finalized instances. This replaces nothing in lichen's trust model: as under BitVMX, the contract is set up by its two parties alone. What it adds is cost at opening (Section 8.4).

The two parties' conduct at setup protects different funds. A hub that garbles wrongly could refuse a valid withdrawal, since Alice could never decrypt; cut-and-choose is what rules this out, so Alice's funds rest on it. Alice's part in setup is small (her keys, her pre-signatures, and which instances to keep), and every secret is the hub's, so the hub's funds rest on the proving phase, where BABE's proofs apply.

One-shot. Revealing labels spends a garbled circuit's security, so a BABE instance settles one claim. A lichen contract has at most one withdrawal, after which it closes; renewal is a new contract with a new setup.

4. The fixed statement

BABE's ciphertext is built from T, and T contains $e(L, \gamma)$, the combination of the public inputs. So the whole public input is fixed at setup. For a RISC Zero proof that input is the claim digest, a hash over the image id and the journal; under BitVMX the journal is $b \,\|\, c \,\|\, \mathcal{D}$, so setup would fix b, which Alice chooses only when she withdraws. The ways round it that we know of:

- *Supply rL at proving time* for the asserted public input only, by envelopes opened with labels the verifier reveals per bit. With a single scalar public input this fails: the points $r\,2^j\,IC$ share one base, so one of them gives away $r\,IC$ and with it the ciphertext for any input. It needs each bit to be its own public input with an independent base, so a custom Groth16 wrap and its own trusted setup, and a garbled SHA-256 between b and RISC Zero's digest.
- *Deferred binding at the cryptographic layer*, as GOAT's researchers propose [8]: a blinding point and further garbled circuits for the dynamic part of the input, one of them about 200 million gates, and a larger challenge transaction.
- *One instance per digit value*: decompose b in base B and set up a fixed statement for each nonzero digit at each position. A dispute then proves each nonzero digit separately; base 16 for a 14-bit amount means 60 setups per contract and up to four disputes.
- *Refresh the statement* whenever Alice's balance changes, which needs contracts in channels so that the refresh is off chain, and a setup per change of balance.
- *Set up only when withdrawing.* Rejected: a hub that declined the setup would leave Alice's tokens without collateral, which breaks the contract's first property, that within its cap Alice trusts the hub for nothing.

We take none of these. Section 5 moves b into the witness instead, so one fixed statement, set up once per contract, serves any amount, and Script reads b from the parties' signatures.

5. Authenticated amounts

5.1. The statement

Let the hub hold, per contract, a one-time signing key pk_H, fixed at setup. The statement proved becomes

$$\exists\, b, w, \sigma_H : \quad \mathrm{Verify}(pk_H, b, \sigma_H) = 1 \;\wedge\; \mathrm{ValidWithdrawal}(c, \mathcal{D}, b, w) = 1,$$

where ValidWithdrawal is lichen's withdrawal statement unchanged (a return of b to the hub with the contract's memo c, under an anchored root of a valid history, with the contract's work on top) and w its witness. The journal holds only values fixed when the contract opens: c, $\mathcal{D}$ and a digest of pk_H. The claim digest, and so BABE's statement, is then fixed at setup, while b is free until Alice withdraws.

5.2. The flow

1. *Request.* Alice asserts b on chain under her one-time key for it, as she does today.
2. *Response.* The hub must sign the same b under pk_H, on chain, within its window. The response's script verifies both signatures and checks, digit by digit, that they sign the same value. If the hub is silent, Alice's timeout pays her the asserted b, as the hub's silence does under BitVMX.
3. *Proof.* Alice proves the statement, using the hub's signature as part of her witness, and wraps the proof.
4. *BABE.* Assert, ChallengeAssert, WronglyChallenged or Failure, as in Section 2.
5. *Payout.* Success pays through the bit outputs, which read b from Alice's signature, as under BitVMX.

5.3. Why the amount is bound

The hub's signature is not a judgement. It means "this is the amount being adjudicated", not "this withdrawal is valid", and an honest hub gives it mechanically for whatever Alice asserted, true or false. Three links bind the amount proved to the amount paid:

- the proof's b is the hub's, because the guest checks the hub's signature;
- the hub's b is Alice's on-chain b, because the response's script compares them digit by digit;
- Alice's b is the one paid, because the bit outputs read her signature, and a second signature from her under the same key forfeits them (`equiv_b`).

Suppose Alice asserts 100 but can prove a withdrawal of only 10. She has the hub's signature on 100 and no valid withdrawal of 100, or a valid withdrawal of 10 and no signature on 10. Neither satisfies the statement.

The authenticator must be the hub. Alice's own signature on b inside the witness would bind nothing: she could sign 10 privately for the proof and 100 on chain for the payout, and the private signature, hidden in a zero-knowledge witness, would never be seen, so the equivocation could not be shown. A key she does not hold, used only on chain, is what carries the on-chain value into a fixed statement. The hub must therefore never sign a second value under pk_H, not even off chain; its key is used once, in the response, and the cooperative fold never touches it.

The payout could check both signatures. Since the hub's signature on b is on chain, each bit output could require both parties' signatures on the same value: Alice's leaf with bit i set,

the hub's with it clear. Neither can then spend with a value the other did not sign, which removes `equiv_b` and Alice's wait δ at about the same script cost.

5.4. The guest

The withdrawal guest takes b and σ_H as private inputs, checks σ_H against pk_H, and commits c, $\mathcal{D}$ and pk_H's digest. The cost of the check depends on the signature's hash. LN-GAP's one-time signatures chain `HASH160`, RIPEMD-160 of SHA-256: up to 150 chain steps for a 32-bit word, and RIPEMD-160 has no accelerator in RISC Zero's zkVM, so a few hundred thousand cycles, comparable to today's whole statement of 389,000 (est.). Chains of SHA-256 instead, which Script can check with `OP_SHA256` at 32 bytes a chain value and the zkVM accelerates, would cost a few thousand cycles (est.). The hub's key should use them.

5.5. Security, as a sketch

The statement cannot say that the hub signed b, only that a valid hub signature on b exists, and one exists for every b; only the hub can compute it. So what binds the amount is not that a false statement cannot be proved but that the prover must *know* a witness: that whoever is paid knows one, which then contains a hub signature, which unforgeability then forces to be on the asserted amount. The pipeline already relies on this: the chain proof's checks of users' signatures bind in the same way.

BABE gives the first step. Its knowledge soundness (Definition 9 of [3]) states that for any NP relation, a prover who gets the Success transaction onto the chain knows a witness, in the sense that an extractor recovers one from its view; it is proved for an honestly run setup (its Definition 16 and Theorem 8). Here the relation is the statement above, through two more layers: Groth16 is knowledge-sound (in the generic or algebraic group model), so the Groth16 proof yields a valid RISC Zero receipt; RISC Zero's STARKs are arguments of knowledge (in the random-oracle model), so the receipt yields the guest's input, (b, w, σ_H). If $b \neq b^*$, the asserted amount, then σ_H is a signature under pk_H on a message the hub never signed, and the hub signed exactly one: a forgery against a one-time signature.

What remains to be written properly:

- BABE proves knowledge soundness for an honestly run setup, not yet for one in which the prover takes part. Alice's part is small and every secret is the hub's (Section 3), so we expect the extension to be routine.
- The hub's signature arrives during the proving phase, after the statement is fixed, and depends on Alice's own assertion: the extraction must hold with the signature in the prover's view, which BABE's definition allows but which should be checked against its proof.
- The composition of extractors across BABE, Groth16 and the STARKs, under BABE's restriction to benign auxiliary input.

6. The transaction graph

The graph follows BABE's (its Appendix A.1), joined to lichen's contract and payout (Figure 1):

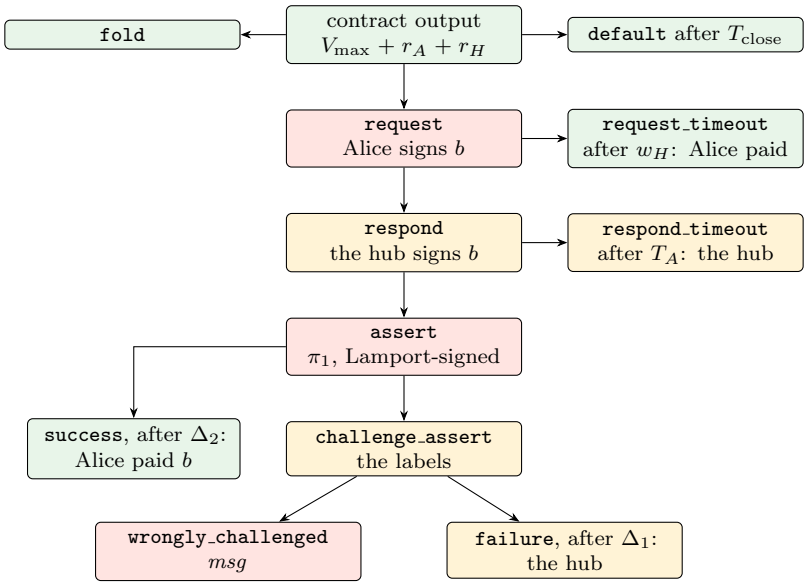


Figure 1: The dispute's graph. Red: Alice's transactions; yellow: the hub's; green: settlements. `success` and `failure` also spend the value output of `respond`; `wrongly_challenged` spends the output `failure` needs, so after it only `success` remains.

transaction	poster	spends, and what it carries or checks
`request`	Alice	the contract output; her one-time signature on b (one word)
`respond`	hub	`request`'s output, within w_H; the hub's signature on b, and Alice's again for the digit-by-digit comparison; creates a value output and a connector
`request_timeout`	Alice	`request`'s output after w_H; the payout, unverified
`assert`	Alice	the connector, before an absolute deadline T_A; π_1 under her Lamport key
`respond_timeout`	hub	the value output and the connector, after T_A: Alice never asserted
`challenge_assert`	hub	`assert`'s second output, within $\Delta_2 - \Delta_1$; the labels, matched to Alice's Lamport signature
`wrongly_challenged`	Alice	`challenge_assert`'s output, within Δ_1; the preimage msg
`failure`	hub	the value output, `assert`'s first output and `challenge_assert`'s, after Δ_1; the hub takes everything
`success`	Alice	the value output and `assert`'s first output, after Δ_2; the payout with its bit outputs

All are pre-signed at opening, about ten transactions, `wrongly_challenged` by Alice alone (Section 7). The payout and its bit outputs are as under BitVMX.

The assertion's deadline. Alice's heavy move, `assert`, needs no window: a late assertion only delays a payout the hub owes anyway, and the hub loses nothing by waiting. So its deadline T_A is absolute, early enough before T_{close} that Δ_2 and the payout fit, and the proof, which needs the hub's signature first, never races a timeout. What remains due within a window is, for Alice,

`wrongly_challenged` (149 vB) and, for the hub, its response and its challenge.

7. Alice's liveness: a keyless tower

Under BitVMX, Alice cannot delegate her moves without her keys. Each move signs, with a one-time key, hashes from her trace at positions the hub's last choice selects, so it cannot be signed in advance; a tower that plays her side holds her move keys, and by equivocating could lose her the contract [2, §10]. Making those moves keyless would need `OP_CAT`. BABE removes the problem rather than shrinking it: Alice's only commitments, b and π_1, are fixed before anything she must answer, and her one answer to the hub is a hash preimage, not a signature. This is a difference of kind, not of count: a gadget with fewer interactive moves would still need someone holding her keys while the dispute runs; here no one does.

What she signs, and when. Two one-time signatures: on b, for `request`, made when she returns b on the L2; and on π_1, the Lamport signature of `assert`, made once she has the proof. The proof needs the hub's signature on b, so the second comes after `respond`, but before the loose, absolute deadline T_A: a second visit, not a race. A prover she does not trust with keys can make the proof; she checks it (one Groth16 verification) and signs π_1.

What the tower holds. The pre-signed transactions, the two signed witnesses, the kept garbled instances (about 89 MiB) and the WE ciphertext, and her proof, which reveals nothing of the witness. From these and the chain it plays everything left to Alice:

- `request_timeout` and `success`: pre-signed, released by their timelocks;
- `wrongly_challenged`: it reads the hub's labels from `challenge_assert`, evaluates the garbled circuit to $r\pi_1$, decrypts with π_2 and π_3, and posts msg.

Her bit outputs wait δ for her own key, as under BitVMX, and nothing races her for them.

Two changes to make. First, in BABE, the leaf `wrongly_challenged` spends checks the prover's signature as well as the hashlock, $\mathsf{HashLock}(h_{msg}) \wedge \mathsf{CheckSig}(pk_P)$, signed when posted (its Appendix A.1). A tower cannot sign then, but the signature should stay. Any spend of the leaf reveals msg and blocks `failure`, which is what Alice wants, but only once it confirms, and the hub knows msg from the start. Without the signature the hub could spend the leaf itself, in a large transaction at a low fee rate, which Alice's spend could replace only by paying more in all (BIP 125's third rule); if the hub's spend does not confirm within Δ_1, it replaces it with `failure`. So Alice makes the signature at setup instead: `wrongly_challenged` is fixed by then, and msg is in its witness, not under its signature. She signs it `ALL|ANYONECANPAY`, as a TRUC (version 3) transaction, and the tower adds msg and a fee input of its own.

Second, fees. The terminals (`request_timeout`, `success`, and now `wrongly_challenged`) are signed `ALL|ANYONECANPAY`, and their broadcaster adds a coin of its own [2, §6]: keyless already. But the links (`request`, `assert`) keep their ids and are bumped through an anchor only their poster can spend, which lichen today locks to the key that also sweeps her winnings. The anchors need a key of their own, guarding nothing but 330 sats each, which she can hand the tower with the witnesses; or she posts both links herself, since neither is due in a window.

A compromised tower can do little. It holds no key, so it cannot sign a wrong move or a second value. That matters here: given her Lamport key, a tower that signed two values of π_1 would let the hub combine their bits into a third, signed bit by bit, and challenge with labels for it; she could not decrypt, and `failure` would take everything. So the tower is not trusted like her own machine, as under BitVMX, but like a Lightning watchtower: several can run at once, strangers to her and to each other, and any one suffices. One caveat: a tower run by the hub holds the signed `wrongly_challenged`, and could post it with *msg* as soon as it posts `challenge_assert`, padded with inputs of its own, to pin it as above. TRUC's limit of 10 kvB on the transaction bounds what an honest tower must outbid, about 100,000 sats at 10 sat/vB. Her wallet, for its part, signs one π_1 only, and keeps the proof it signed.

8. Estimates

8.1. Transactions and disputes

transaction	size
`request`: one signed word	∼0.4 kvB (est., from lichen's measured ∼250 vB a word)
`respond`: two signed words and the comparison	∼0.6 kvB (est.)
`assert`	9.24 kvB (BABE)
`challenge_assert`, one label set (soldered)	17.4 kvB (BABE)
`challenge_assert`, four label sets	∼42 kvB (est.: one set is about $17.4 - 9.2 = 8.2$ kvB)
`challenge_assert`, one secret set by adaptor signatures (VSSS), in 2-bit digits	∼21 kvB (est., Section 8.5), with `assert` ∼7.8
`wrongly_challenged`	0.15 kvB (BABE); ∼0.21 with a fee input
a timeout, `failure`, `success`	∼0.3–0.8 kvB (est.)
the payout and Alice's sweep of her bits	∼3 kvB (lichen)

exit	BABE (est.)	BitVMX [2]
the hub present: a fold	0.3 kvB	0.3 kvB
the hub gone	∼3.8 kvB: request, its timeout, the payout	23.1 kvB
the hub refuses a true claim to the end	∼31 kvB in six transactions (∼56 unsoldered)	∼120 kvB in 23
the hub disproves a false claim	∼28 kvB (∼53 unsoldered)	100.4 kvB
the worst case	the same: there is no separate case for reads	200–212 kvB, a read lie
Alice's transactions, those due in a window	4 (request, assert, wrongly challenged, success), one	13, twelve
keys Alice's tower must hold	none, but an anchor key (Section 7)	her move and hop keys, and her input words'
pre-signed per contract	about 10	141

Most of the gain against a vanished hub comes from not publishing the proof's input words before the hub moves; BitVMX could recover much of it by demanding them only from a hub that is present [2].

8.2. Congestion

The main paper's rule, $Ns \leq w \cdot 1\,\text{MvB}$ for N contracts whose move of s bytes is due in a window of w blocks, must hold for every window. Comparing gadgets by one window would be unfair to the one with fewer: a BitVMX dispute is twelve windowed moves on each side, one after another, so a hub safe at a given window under BitVMX asks for a dispute twelve windows long. The fair basis is the dispute's length T for a given N, with each window sized to its move: $N \leq T \cdot 1\,\text{MvB}/\sum_i s_i$, summed over the defender's windowed moves [2, §9].

	BABE, Alice against a hostile hub	BABE, the hub against hostile users	BitVMX, the binding side
windowed moves	`wrongly_challenged`, 0.21 kvB	the response 0.6 and `challenge_assert` 17.4 (42 unsoldered)	as built, Alice: twelve windows sized to her proof; with a window per move, the hub
$\sum_i s_i$	0.21 kvB	18 kvB (43)	302 kvB; 121 kvB
contracts safe with a dispute of a day	∼690,000	8,000 (3,400)	480; 1,190
of a week	—	56,000 (23,700)	3,300; 8,350
the dispute for 60,000 contracts	13 blocks	7.5 days (18 days)	126 days; 50 days

Hostile users bind under BABE. For a dispute of the same length, BABE holds about 6.7 times as many contracts as BitVMX with a window per move (2.8 unsoldered), and 17 times as many as BitVMX as built (7 unsoldered). A hostile hub stops mattering, since Alice's only windowed move is 149 vB; were her assertion given a window rather than a deadline, her sum would be 9.5 kvB, about 106,000 contracts for a week. The hub's response, 0.6 kvB in w_H, is part of its

sum: 60,000 contracts need w_H of at least 36 blocks. Chunking BitVMX's path into separate windows would not change these figures, since it shrinks the path's largest move but not its sum.

8.3. Reserves and time

A reserve must cover the other side's fees in a dispute it loses. Alice's side is about 12.5 kvB (request, assertion, the preimage, her payout), against about 117 under BitVMX, so the hub's reserve per user falls about tenfold: about 125,000 sats at 10 sat/vB rather than 1.2 million, and the contract suits balances of some tens of thousands of sats rather than half a million. The hub's side is about 18 kvB (43 unsoldered). A hostile hub stretches a dispute to about $w_H + \Delta_2$ blocks after Alice's assertion, about 170 blocks with $w_H = 12$, $\Delta_2 = 156$, $\Delta_1 = 12$, against about 430 under BitVMX with the windows the main paper uses; a vanished hub's users wait w_H, as now.

8.4. Setup per contract

The new cost. At BABE's operating point (2,268 instances, 4 kept), on a 16-thread laptop (BABE):

item	per contract
cut-and-choose, both roles	1 min 37 s, 1.8 GB at peak
soldering, for one label set on chain	745 s more, the hub's
what Alice stores	∼89 MiB (4 instances of 22 MiB); perhaps ∼2 MiB with Duty-Free Bits' 45-fold smaller encoding [7], unverified for this use
transferred from the hub to Alice	at least the kept instances, ∼89 MiB (est.)
a hub opening 10,000 contracts	∼270 laptop-hours, ∼2,300 with soldering: parallel, and again at each renewal

Fewer instances trade setup for chain space: keeping 10 of 78 sets up in 6 s but, unsoldered, puts 10 label sets on chain. Nearly all of the soldered setup's 14 minutes is the soldering, not the cut-and-choose.

8.5. Setup by verifiable secret sharing

Fourteen minutes of computation per contract, paid again at every renewal, is probably more than opening a contract can bear; forty seconds is not. Glock [6] replaces soldering with verifiable secret sharing (VSSS), and BABE's authors have prototyped that layer for BABE's 508 input bits (BABE, §9.4), without a transaction design. This is the setup we would want, and what it needs on chain is sketched below.

Why BABE keeps 4 of 2,268. Cut-and-choose fails only if every kept instance is bad and every opened one good, so the error is $\binom{N}{M}^{-1}$, held to 2^{-40}. Unsoldered, each kept instance puts a label set on chain; soldered, the proof's time grows with M (745, 866 and 969 s for 4, 5 and 6, BABE). Either way M must be small, and a small M forces a large N.

Secret sharing reverses that. The hub shares each input label, for each bit and each value, across the N instances, with threshold $N-M+1$, and publishes commitments to the polynomials (Feldman's, over secp256k1). Alice checks the opened instances' shares against them. The labels of a free-XOR garbling are related ($L^1 = L^0 \oplus \Delta$), so they cannot themselves be independent shares: the shared values are secrets of their own, which each instance maps to its labels through a small table, $H(s) \oplus L$. At proving time one secret per bit goes on chain, $f(0)$ of its polynomial; with the $N - M$ opened shares, Alice interpolates the secrets, and so the labels, of every kept instance. The value Alice did not assert stays hidden, since only $N - M$ points of its polynomial are known. The chain carries one label set whatever M is, and the cost moves to N, the shares to commit and check. So the right choice is now a small N:

(N, M)	setup, both roles	Alice stores and receives
(2,268, 4), soldered	∼14 min (BABE)	∼89 MiB
(78, 10), VSSS	∼40 s (BABE's estimate: 33 s for the VSSS layer)	∼220 MiB
(124, 8), VSSS	∼1 min (54 s for the layer)	∼180 MiB
(181, 7), VSSS	∼1.5 min (84 s for the layer)	∼155 MiB

That is about twenty times faster than soldering, and a hub opening 10,000 contracts spends about 110 laptop-hours rather than 2,300. The bill moves to storage and transfer: Alice keeps every finalized instance, 22 MiB each, since she cannot know which one is honest. Glock's even split, about 44 instances with 22 kept, would be the wrong end of that trade here, at nearly 500 MiB. Duty-Free Bits' 45-fold smaller encoding [7], if it holds for these instances, would bring even $M = 10$ to about 5 MiB.

On chain: adaptor signatures. The secret posted on chain must be the one Alice checked against the commitments, and the chain must check that. A hashlock cannot: tying a hash to a committed point sG needs a proof, which is what soldering is. Glock uses adaptor signatures instead. At setup Alice gives the hub, for each bit i and value β, a pre-signature on `challenge_assert` under her own one-time key $A_{i,\beta}$, adapted to the committed point $C_0 = f_{i,\beta}(0)\,G$. The script selects $A_{i,\beta}$ by her Lamport signature, which the hub copies from `assert`, and the hub can satisfy it only by completing her pre-signature, which hands her $f_{i,\beta}(0)$. The selection matters both ways: the other value's secret would leave Alice unable to decrypt, and both values' would let her evaluate the circuit on points other than π_1. In Glock the garbler is also the prover, so its adaptor signatures are its assertion; in BABE the prover asserts and the verifier reveals, so both are paid for.

Each secret is the adaptor of exactly one pre-signature, under a key used for nothing else. Nothing is adapted twice to one point, so the setting is adaptor signatures' basic one-time case; one secret serves all M kept instances, but by interpolation off chain, not through M signatures. The pre-signatures are all on one sighash, under different keys, which BIP340's challenge includes. Alice makes them once the kept instances are chosen: their hashlocks are in `challenge_assert`'s output, and so in the sighash.

Two bits a step. Bit by bit, this does not fit in one input. Tapscript limits an input's initial stack to 1,000 elements (BIP 342), and each bit needs two, the hub's signature and Alice's preim-

age: 1,016 for π_1's 508 bits, 1,017 with the hub's own signature. Splitting `challenge_assert` over two inputs would do, but a better fix also saves bytes. Alice signs π_1 in 2-bit digits, revealing one of four independent preimages per digit: Lamport over a larger alphabet, which, unlike Winternitz's chains, needs no checksum. The hub shares one secret per digit and value, which each instance maps to both bits' labels. The polynomials number the same, $254 \times 4 = 508 \times 2$, as do the keys and hashes in the leaf, but the hub's signatures and Alice's preimages halve, to 254 each, and the stack holds 509 elements. Per digit the leaf is about 237 bytes (four hashes, four keys and the selection) and the witness 86 (a signature and a preimage):

	bit by bit	2-bit digits
`challenge_assert`'s stack	1,017 elements: two inputs	509
`challenge_assert`	∼26 kvB (est.)	∼21 kvB (est.)
`assert`	9.24 kvB (BABE)	∼7.8 kvB (est.)
a false claim disproved	∼37 kvB	∼30 kvB
a true claim refused to the end	∼39 kvB	∼33 kvB
the hub's windowed sum	∼27 kvB	∼22 kvB
contracts for a week's dispute	37,000	47,000

Soldered, the same rows are 17.4, 9.24, 28, 31 and 18 kvB with 56,000 contracts; ten label sets unsoldered put about 91 kvB in `challenge_assert`. `assert`'s figure scales BABE's by our count of script and witness, 118 bytes a digit against 70 a bit. Wider digits do not pay: at three bits the keys and hashes per bit rise from two to 8/3. The pre-signatures, 1,016, are a small transfer, about 100 KB.

The keyless tower survives. Alice's keys $A_{i,\beta}$ are used once, at setup, to make the pre-signatures, and never again. To read the secrets the tower needs, beyond what Section 7 lists, her pre-signatures (32 KiB of their scalars) and the opened shares of both values (about 2 MiB at $(78, 10)$), none of them a key, and it extracts and interpolates as Alice would. The kept instances it holds are now M of them, about 220 MiB at $(78, 10)$ rather than 89.

9. With a counter-proof

lichen's planned exit for a stalled or censoring sequencer lets Alice withdraw from anchored state by publishing her notes' nullifiers, and lets the hub answer with a *counter-proof*, a later anchored spend of one of them. Under BABE, two shapes:

- *Inside Alice's statement.* The hub's response carries its counter, or "none", under its signature; Alice's statement adds "and the hub's counter, if any, is invalid". One BABE instance; the response grows by the counter's bytes when used, about 3 kvB (est.), and Alice's guest must verify a Groth16 proof (cycles unmeasured).
- *A separate game*, roles reversed: the hub proves its counter in a second BABE instance garbled by Alice. Setup doubles, and Alice's heaviest windowed move becomes her own `challenge_assert`, so the hostile-hub side falls to the hostile users'.

Either way, Alice's request also carries the nullifiers' digest, which the hub countersigns: about

2 kvB more on each (est.).

10. Open questions

- The security argument of Section 5.5, written as a proof.
- A proof of robustness under cut-and-choose, against a hub that garbles wrongly. It is BABE's to give, but lichen's users rely on it.
- The secret-sharing setup of Section 8.5: its transaction design, built and measured, and BABE's 40-second estimate confirmed end to end; its security argument with the shares and adaptor signatures in place of soldering.
- What Duty-Free Bits' smaller encoding means for what Alice must store and receive.
- The response transaction's script and size, and the guest's signature check, built and measured.
- Whether BABE's authors, or GOAT's, see a problem with moving the amount into the witness this way.

References

[1] waxwing. Lichen: L2s in channels enforced by Bitcoin. Version 0.2, October 2026. https://github.com/AdamISZ/lichen

[2] waxwing. Lichen over BitVMX: the dispute, as built and measured. Version 0.1, October 2026. https://github.com/AdamISZ/lichen

[3] S. Garg, D. Kolonelos, M. Sergeevitch, S. Sridhar and D. Tse. BABE: verifying proofs on Bitcoin made 1000x cheaper. ePrint 2026/065, revised October 2026. https://eprint.iacr.org/2026/065

[4] S. Garg, D. Kolonelos, G.-V. Policharla and M. Wang. Threshold encryption with silent setup. CRYPTO 2024.

[5] L. Eagen and Y. T. Lai. Argo MAC: garbling with elliptic curve MACs. ePrint 2026/049. https://eprint.iacr.org/2026/049

[6] L. Eagen. Glock: garbled locks for Bitcoin. ePrint 2025/1485. https://eprint.iacr.org/2025/1485

[7] A. Khambhati, S. Bhattacharya and D. Heath. Duty-free bits. ePrint 2026/476. https://eprint.iacr.org/2026/476

[8] GOAT Network research. Dynamic public inputs in BABE (deferred binding). HackMD, 2026. https://hackmd.io/@goatresearch/B1N9g2uOee

[9] waxwing. Variable payout amounts for BABE, as applied to Lichen. Delving Bitcoin, October 2026. https://delvingbitcoin.org/t/variable-payout-amounts-for-babe-as-applied-to-lichen/2963

[10] S. D. Lerner, R. Amela, S. Mishra, M. Jonas and J. Álvarez Cid-Fuentes. BitVMX: a CPU for universal computation on Bitcoin, 2024. https://arxiv.org/abs/2405.06842

[11] RISC Zero. The RISC Zero zkVM and its Groth16 receipts. https://dev.risczero.com